Ders 9 Genişletilmiş Notu: Döngüler: For ve While Notasyonu, Faktöriyel ve Basamak Sayısı

Ders 9 Genişletilmiş Notu: Döngüler: For ve While Notasyonu, Faktöriyel ve Basamak Sayısı

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Üçgen Türü Problemi
- 2. Dik Üçgen Kontrolü ve Verimlilik
- 3. Döngülere Giriş: For mu, While mı?
- 4. For Döngüsünün Notasyonu
- 5. Faktöriyel Hesabı
- 6. While Döngüsünün Notasyonu
- 7. Bir Sayının Basamak Sayısı
- 8. Negatif Sayılar ve Alternatif Koşul

1. Üçgen Türü Problemi

Ders, önceki dersten kalan bir problemle başladı: kullanıcıdan üç kenar uzunluğu alınacak, üçgenin eşkenar, ikizkenar ya da çeşitkenar olduğu bulunacak. Hoca burada akış diyagramı çizdirmekten önce öğrencilerin kağıt üzerinde probleme nasıl yaklaştığını dinlemek istedi: “şuna baktım, bunu kontrol ettim, böyle olursa şuna baktım” biçiminde sözlü bir algoritma.

1.1 İlk Akla Gelen Çözüm ve İki Sorunu

Bir öğrenci çözümünü şöyle anlattı: önce kolay olanla başlıyor ve “A, B ve C birbirine eşit mi?” diye bakıyor, eşitse eşkenar diyor. Değilse “A eşit C mi, ya da A eşit B mi, ya da B eşit C mi?” diye bakıp ikizkenar olup olmadığına karar veriyor. O da değilse üçünün birbirine eşit olmadığını kontrol edip çeşitkenar diyor. Hoca, bunun pek çok öğrencinin aklından geçen çözüm olduğunu tahmin ettiğini söyledi ve iki kritik noktaya dikkat çekti.

Birincisi, karşılaştırma operatörleri (eşittir, eşit değildir, büyüktür, küçüktür, büyük eşit, küçük eşit) iki operand alır. Bir karar kutusunda “A büyük B mi?” diye sorabilirsiniz, ama aynı kutuda üçlü, dörtlü, beşli karşılaştırma yapamazsınız.

> **Hocanın vurgusu:** Bir if içinde aynı anda `a < b, b < c` ya da `a = b = c` diyemezsiniz. Bir operatör iki operand alır (birkaç istisnası olduğunu da belirtti). Burada önemli olan, karşılaştırmaların ikili olmasıdır.

İkincisi, gereksiz kontroldür. Bir şeye baktıktan sonra geriye tek bir seçenek kaldıysa onu yeniden sormaya gerek yoktur. Kontrol etmeden o sonuca varabilirsiniz. Eşkenar ve ikizkenar elendiyse üçgen zaten çeşitkenardır.

1.2 Akış Diyagramı

Hoca çözümü öğrencilerle birlikte, ekranda adım adım çizdi. Kullanıcıya önce kenar uzunluklarını girmesi için bir mesaj gösterilir, sonra üç girdi (A, B, C) okunur. Ardından öğrencilerin

önerileriyle iç içe kararlar kuruldu. Hocanın burada belirttiği gibi, bu iki koşulu tek bir if içinde sormak da mümkün, ama bu ilerleyen derslerin konusu. Şu anda iç içe iki if kullanılıyor: bir koşul sağlanıyorsa içeride bir koşul daha soruluyor.

```
1. START
2. Mesaj: "Kenar uzunluklarını giriniz"
3. Girdi: A, B, C
4. A = B mi?
   Doğruysa:
     B = C mi?
       Doğruysa: çıktı "eşkenar"
       Yanlışsa: çıktı "ikizkenar"
   Yanlışsa:
     B = C mi?
       Doğruysa: çıktı "ikizkenar"
       Yanlışsa:
         A = C mi?
           Doğruysa: çıktı "ikizkenar"
           Yanlışsa: çıktı "çeşitkenar"
5. Tüm kollar içten dışa doğru birleşir
6. STOP
```

Buradaki önemli nokta en içteki adımdır. Bir öğrencinin belirttiği gibi, A ile B eşit değil ve B ile C de eşit değilse hemen çeşitkenar demek yanlıştır, çünkü A ile C eşit olabilir. Bu yüzden A = C de sorulur. Üç kontrol de yanlış çıktığında "A, B ve C birbirinden farklı mı?" diye bir daha sormaya gerek yoktur.

Çözümde tek bir eşkenar durumu, tek bir çeşitkenar durumu ve üç ikizkenar durumu vardır: A'nın B'ye, B'nin C'ye ve A'nın C'ye eşit olması. Kenar uzunluklarıyla tür belirlerken bu üç eşitlik durumu da bağımsız olarak ortaya çıkabildiği için üç kontrol de gereklidir.

1.3 If Kollarının Birleştirilmesi

Hoca çizerken if kollarını özellikle bağladığını gösterdi: önce en içteki ifin kolları birleşir, sonra bir üstteki if, en sonunda en dıştaki if birleşir ve akış tek bir STOP'a gider.

Soru: İki farklı koşuldan iki ok çıkarıp daha güzel görünmesi için bunları tek bir çıkış kutusuna yönlendirebilir miyiz?

Cevap: Akış diyagramının güzel görünmesinin hiçbir önemi yoktur. Kurallar vardır ve diyagram bu kurallara uygun notasyonla çizilir. İki farklı ifin true kolunu tek bir output kutusunda birleştiremezsiniz. Birden fazla STOP da kullanamazsınız. Bir kola, gerekiyorsa alt alta birden fazla çıktı konabilir, ama farklı kollar gidip tek bir çıkış kutusunda birleşemez.

> **Hocanın vurgusu:** Bir iften iki kol çıkıyorsa bu kollar önce birleşir, sonra devam eder. O if yukarıdaki bir ifin true kolundaysa, oradan çıkan kol da onun false koluyla birleşir.

1.4 Algoritmanın Analizi

Analiz için önce tüm değişkenler (A, B, C) ve çıktı bir tabloya yazılır, sonra farklı durumlar denenir. Bir öğrenci ilk örnek olarak 3, 5 ve 9 kenarlarını önerdi. Başka öğrenciler bu üç sayının üçgen kuralına uymadığını hatırlattı, hoca da teşekkür edip örneği 3, 5, 7 olarak değiştirdi.

> **Ek bilgi (derste anlatılmadı):** Derste adı geçen üçgen kuralı büyük olasılıkla üçgen eşitsizliğidir: herhangi iki kenarın toplamı üçüncü kenardan büyük olmalıdır. 3, 5, 9 için $3 + 5 = 8 < 9$ olduğundan bu kenarlarla üçgen kurulamaz. 3, 5, 7 için ise $3 + 5 = 8 > 7$ olur.

Aşağıdaki tabloda 3, 5, 7 izini derste bir öğrenci anlattı. Öteki satırların izi ise hocanın yalnızca girdilerini yazdığı durumlardan kuruldu (ara adım eklendi).

A	B	C	İzlenen kontroller	Çıktı
3	5	7	$3 \neq 5$, $5 \neq 7$, $3 \neq 7$	çeşitkenar
2	2	2	$2 = 2$, $2 = 2$	eşkenar
3	3	4	$3 = 3$, $3 \neq 4$	ikizkenar
4	4	2	$4 = 4$, $4 \neq 2$	ikizkenar

> **Ders sonrası not:** Altyazıda geçen biçimiyle 3, 3, 4 ve 4, 4, 2 örneklerinin ikisi de A = B koluna gider. B = C ve A = C kolları bu sayılarla denenmiş olmaz. Hocanın ekrana yazdığı sıralama büyük olasılıkla farklıydı. Algoritmayı sınarken üç ikizkenar kolunun da en az birer kez izlenmesi gerekir.

Hoca problemin bu halini dik üçgeni dışarıda bırakarak bitirdi ve şu bağlantıyı kurdu: önceki sorularda daha çok aritmetik işlem yapılmıştı, burada ise yalnızca doğru sorular sorarak üçgenin türüne karar verildi. Hayatta pek çok problem doğru koşulları belirleyerek çözülür. İlerleyen problemlerde if yapıları ile aritmetik işlemler daha fazla iç içe geçecek.

2. Dik Üçgen Kontrolü ve Verimlilik

Sonra problem biraz zorlaştırıldı: üçgenin dik olup olmadığı da bulunacak. Kural bellidir: Pisagor teoremine göre herhangi bir kenarın karesi, diğer iki kenarın karelerinin toplamına eşit olmalıdır. Hangi kenarın en uzun olduğu bilinmediğinden üç olasılık vardır:

$$a^2 = b^2 + c^2 \qquad b^2 = a^2 + c^2 \qquad c^2 = a^2 + b^2$$

Hoca, bunlardan biri sağlanmıyorsa diğerlerinin de sağlanmadığını söyleyemeyeceğimizi belirtti. Biri sağlanmazsa öteki koşullara da bakmak gerekir. Bir koşul sağlandığı noktada ise akış, kalan kontrollere geçecek biçimde devam ettirilebilir.

2.1 Öğrencilerin Yaklaşımları

Derste üç yaklaşım dinlendi:

- **Önce Pisagor kontrolü:** Üç kenarın karesi hesaplanır, önce $a^2 = b^2 + c^2$ sorulur, sonra öteki iki eşitlik sorulur. Hiçbiri sağlanmıyorsa ilk çözümdeki A = B kontrolüne bağlanılır.
- **Önce Pisagor, içinde ikizkenar ayrımı:** $a^2 = b^2 + c^2$ doğruysa B = C mi diye de bakılır. Doğruysa “ikizkenar dik üçgen”, değilse “dik üçgen” yazılır. Yanlış kolda öteki iki Pisagor eşitliği denenir, sonra A = B ile devam edilir.
- **Önce tür, sonra Pisagor:** Çeşitkenar, ikizkenar, eşkenar ayrımı aynen yapılır. Her bir sonuç ile STOP arasına ayrıca dik üçgen kontrolü eklenir. Böylece hem “ikizkenar” hem de “dik üçgen” çıktısı verilebilir.

Hoca hepsinin uygulanabilir bakış açıları olduğunu söyledi, ama asıl soruyu üçüncü yaklaşım üzerinden sordu.

2.2 Kontrol Nereye Konmalı?

Soru (hoca): Bir üçgen eşkenarsa dik üçgen olabilir mi?

Cevap: Olamaz.

Bu yüzden önce üçgenin türünü belirleyip dik üçgen kontrolünü sonra yapmak, işlem açısından daha avantajlı görünüyor. A = B ve B = C ile eşkenar sonucuna varıldığında artık $a^2 = b^2 + c^2$ sorulmaz. Bir değil, üç kontrolden kurtulmuş olursunuz. Dik üçgen kontrolü en öne konsaydı, gelen üçgen eşkenar olsa bile her durumda yapılırdı. Hoca, tam analizini yapmadığını ve kötü durumda yine aynı sayıda işlem yapılabileceğini de ekledi.

Bir öğrenci buna itiraz etti: kontrolü sona koyarsak eşkenar dışındaki her sonucun arkasına, yani üç ikizkenar ve bir çeşitkenar koluna, tekrar tekrar eklememiz gerekmez mi? Başa konsa bir kez yazılacak, arkaya konunca dört kez yazılacak. Hoca bunun iyi mi kötü mü olduğunu sınıfa sordu.

Bir öğrenci önemli bir ayrıntıyı söyledi: ikizkenar kollarının altında tek bir koşul yeterlidir. Örneğin A ile B'nin eşit olduğu biliniyorsa yalnızca $c^2 = a^2 + b^2$ sorulur.

> **Ek bilgi (derste anlatılmadı):** Bu tek kontrolün yetmesinin gerekçesi büyük olasılıkla şudur: dik üçgende en uzun kenar hipotenüstür. A = B ise bu iki kenardan biri hipotenüs olamaz, çünkü öteki kenarla eşittir. Geriye hipotenüs adayı olarak yalnızca C kalır.

2.3 Kodun Uzun Olması, Daha Çok İşlem Demek Değildir

Öğrenciler tartışmayı şöyle sonuçlandırdı: diyagramda daha çok işlem varmış gibi görünse de bilgisayar toplamda daha az işlem yapar. Programın kodu büyür, ama daha hızlı çalışır. Hoca da buna katıldı. Her kolun arkasına kontrol koymak daha fazla işlem yaptırmaz, yalnızca kodu ve diyagramı uzatır. Tekrar eden kontroller yazılır, ama sonuçta daha fazla işlem yapılmaz. Hoca bu kavramların zaman içinde oturacağını, ilk etapta tam anlaşılmazsa dert edilmemesini de söyledi.

> **Hocanın vurgusu:** Kod uzunluğu ile yapılan işlem sayısı farklı şeylerdir. Bazen daha fazla kod yazarak, diyagramda daha fazla çizerek, daha az kontrol yapan, karmaşıklığı daha düşük ve daha verimli algoritmalar tasarlayabilirsiniz. Önce doğru sonuç veren algoritmayı tasarlayın. Sonra kendinize sorun: en kötü ihtimalle kaç kontrol yapıyorum, bu işlem ya da karşılaştırma sayısını azaltabilir miyim?

2.4 Girdinin Sıralı Olduğunu Varsaymak

Soru: Kullanıcıdan kenarları küçükten büyüğe sıralı girmesini istersek yalnızca $c^2 = a^2 + b^2$ koşuluna bakmak yeter. Bu kabul edilir mi?

Cevap: Kullanıcıdan düzenli, sıralı bilgi istemek yanlış bir düşünce değildir. Bilginin sıralı olması ya da önce sıralanması, görülecek pek çok algoritmada işi çok kısaltır. Ama bu tür problemlerde kullanıcıyı buna zorlayamayız. Problemde sıralı verildiği açıkça belirtilmedikçe bunu varsayarak çözüm yapılmaz.

Soru: Kullanıcının verdiklerini önceden sıralasak daha hızlı işlem yapma ihtimalimiz olur mu?

Cevap: Bu problem için hayır, çünkü sıralamanın da bir maliyeti vardır. Başka problemlerde önce sıralayıp sonra çözmek daha doğru olacaktır. Bu konunun ağırlıklı olarak sonraki dönemlerde görüleceğini söyledi.

Bir öğrencinin ifin içinde "and" operatörünün kullanılıp kullanılamayacağı sorusunu ise hoca sonraya bıraktı.

3. Döngülere Giriş: For mu, While mı?

Hoca, normalde uygulama yapılan bu saatte önceki haftadan kalan konuya geçeceğini söyleyip döngülere başladı ve bir soru sordu: if ve dört işlem yetmiyor mu, bilgisayarda neden döngü diye bir yapı var? Cevap tektir: aynı işlemi defalarca yapmak istiyorsanız döngülerden yararlanırsınız.

İki temel döngü tipi vardır: **for** döngüsü ve **while** döngüsü. While döngüsünün alt başlıkları da vardır: while'ın kendisi, do while ve repeat until. Bu kullanımlar dilden dile değişir.

Hoca for döngüsünün hangi koşulda kullanılacağını sordu ve şu cevapları değerlendirdi:

- "Bir şey doğru olduğu sürece altındakini tekrar eder." Doğru değil.
- "Bir işlemi listedeki bütün elemanlara uygulamak için kullanılır." Doğru değil.
- "Belirlediğimiz bir değişken belirlediğimiz sayıya gelene kadar döngüyü tekrar başlatır." Tam olarak değil, ama daha doğru.
- "Başlangıcı belli bir değişken bir koşulu sağladığı sürece tekrar eder." Doğru değil.

Doğru cevabı hoca verdi:

> **Hocanın vurgusu:** Adım sayısı belli olan işlerde for döngüsü, adım sayısı belli olmayan işlerde while döngüsü kullanılır. İkisi arasındaki temel fark budur.

Adım sayısının belli olması, onun sabit bir sayı olması demek değildir. Adım sayısı girdiyle ilişkili olsa bile her çalıştırmada o girdi kadar olacağını biliyorsanız for kullanırsınız. Yaptığınız işin kaçıncı adımda sonlanacağını bilemiyorsanız ve bu verilen girdiye göre değişkenlik gösteriyorsa while kullanırsınız. Hoca bu ayrımın örneklerle yerine oturacağını söyledi.

4. For Döngüsünün Notasyonu

Hoca, her sene kullandığı bir örneğe, faktöriyele geçmeden önce for döngüsünün akış diyagramındaki gösterimini ekrana çizdi. Çizimin dokuz noktasını numaralandırdı ve anlamlarını tek tek sordu. Örnekte `I` indisi 1'den başlıyor, birer artıyor ve 100'e kadar gidiyordu.

1. Döngüye giriş: akış diyagramında kalınan yerden for döngüsüne girilen kısım.
2. Döngünün her turunun başladığı nokta.
3. Döngünün bir turunun bittiği, koşula geri dönülen kısım.
4. Döngüden çıkış: koşul artık sağlanmaz, adım sayısı bitmiştir ve akış fordan ayrılarak devam eder.
5. İşlem kutusu: her turda gerçekleştirilen işlemler.
6. END kutusu: her turun sonlandığını sembolik olarak gösterir. Teorik gösterim budur, ama kullanılması zorunlu değildir.
7. Artış miktarı: indisin kaçar kaçar değişeceği. Örnekte `I` her seferinde bir artar.
8. İndisin ilk değeri. Örnekte `I` ilk değeri 1'dir.
9. İndisin alacağı son değer. Örnekte 100'dür.

Yedinci nokta için bir öğrencinin "kaç kere yapacağımız" önerisi kabul edilmedi. Doğrusu "kaçar kaçar artacağı"dır. Girişi 1, çıkışı 4 olarak akılda tutabilirsiniz. Bu örnekte döngü 1'den başlar, son değer olan 100 de dahil olmak üzere 5 numaralı kutudaki işleri arka arkaya 100 defa yapar. Adım sayısı bellidir.

5. Faktöriyel Hesabı

Problem: verilen n sayısının faktöriyelini hesaplayan algoritmanın akış diyagramını çiziniz.

5.1 Önce Kağıt Üzerinde

Hoca, algoritmayı kurmadan önce işi kağıt üzerinde nasıl yaptığımızı düşünmemiz gerektiğini söyledi: önce kafanızda tasarlayın, sonra algoritmaya dökün. 3 faktöriyel için $3 \times 2 \times 1$, 4 faktöriyel için $4 \times 3 \times 2$ yazılır. Her adımda sayıyı bir sonraki (ya da bir önceki) sayıyla çarpar ve elde ettiğimiz değeri saklarız. Genel olarak:

$$n! = n \times (n-1) \times (n-2) \times \cdots \times 1$$

5.2 Adım Sayısı Belli mi?

Algoritmada dışarıdan n alındıktan sonra hoca dersin en önemli sorusunu sordu: burada adım sayısı belli mi?

Soru (hoca): Burada adım sayısı belli mi?

Cevap (öğrenci): Hayır, girdiğimiz sayıya göre değişiyor.

Hoca bu algının değiştirilmesi gerektiğini söyledi. Bir başka öğrenci "kullanıcı girdikten sonra belli" dedi ve hoca bunun genelleştirilmesini istedi. Doğru cevap şudur: istenen değere ulaşmak için bu işin n kere yapılması gerekir. Değişkenin adı n, k ya da z olabilir, ama iş girilen değer kadar yapılır. Bir seferinde n kere, bir seferinde $n/2$ kere, bir seferinde $n-2$ kere gibi bir durum yoktur.

Buradan şu sonuç çıkar: for döngüsünün başlangıç noktası, bitiş noktası ve artım ya da azaltım miktarı sabit olmak zorunda değildir, değişken olarak da verilebilir. Bu değişkenin değerini kullanıcı ya da algoritma belirlediği için adım sayısı yine bellidir. Hoca farkın while örneğinde daha iyi anlaşılacağını belirtti.

5.3 Azalan For ile Çözüm

İndis için genelde `I`, `J`, `K` kullanılır. Öğrencilerin önerileriyle döngü şöyle kuruldu: `I` değeri `N`'den başlar (böyle olmak zorunda değildir), adım -1'dir, son değer 1'dir.

Döngü içinde çarpma yapılacaktır. Ama bunun için önce sonucu tutan bir değişken gerekir. Hoca burada "tanımlamak" kelimesini kullanmak yerine, bir değişken kullanmamız ve ona ilk değer vermemiz gerektiğini söyledi. Değişkenin adı `sonuc` ve ilk değeri 1 seçildi. Döngü içinde `sonuc = I * sonuc` yazılır. `I`'yı eksiltmek için ayrıca bir şey yazılmaz, bunu döngünün kendisi yapar.

```
1. START
2. Girdi: N
3. sonuc = 1
4. for I ← N, adım -1, son değer 1:
     sonuc = I * sonuc
5. Çıktı: sonuc
6. STOP
```

Hoca derste `sonuc` atamasını sonradan araya yazdığını, normalde bunun ayrı bir kutu olarak gösterilmesi gerektiğini ve öğrencilerin bunu kutu olarak eklemesini istedi.

5.4 I Yerine N Yazılabilir mi?

Bir öğrenci işlemde `I * sonuc` yerine `N * sonuc` yazılmasını önerdi. Başka bir öğrenci bunun doğru olmadığını açıkladı: 9 girildiğinde her seferinde 9 ile çarpılır, yani 9, 8, 7 diye değil 9, 9, 9 diye gidilir. Hoca da doğruladı: `I` yerine `N` yazmak, sonucu her turda aynı değerle çarpmaktır. Bir sayının n'inci kuvveti hesaplanırken bu yapılabilir. Faktöriyelde ise çarpılan değer her turda bir azalır, bu yüzden ilk yazılan `I` doğrudur.

5.5 Analiz: N = 5

Hoca doğru yapıp yapmadığımızı anlamanın yolunun analiz olduğunu hatırlattı: tıpkı üçgen sorusundaki gibi değer verip denemek. Analizde kullanılacak olan yalnızca değişkenlerdir: `N`, `sonuc` ve `I`. Başka hiçbir şey düşünmeye gerek yoktur. $N = 5$ için, `N` değişmediği için tabloya yazılmaz; başlangıçta `sonuc` = 1'dir.

Tur	`I`	`I >= 1`	İşlem	`sonuc`
1	5	evet	5×1	5
2	4	evet	4×5	20
3	3	evet	3×20	60
4	2	evet	2×60	120
5	1	evet	1×120	120
çıkış	0	hayır		120

$I = 0$ iken $0 \geq 1$ sağlanmaz, döngüden çıkılır ve ekrana 120 yazılır. Algoritma 5 faktöriyel için doğru çalıştı.

5.6 Uç Durumlar: 0 ve Negatif Sayılar

Hoca 5 dışında hangi durumların denenmesi gerektiğini sordu. Öğrenciler 0'ı önerdi. Bir öğrenci de hatırladığı kadarıyla sıfır faktöriyelin olmadığını, sonucun sıfır olması gerektiğini söyledi. Hoca önce doğru değerler verildiğini varsayarak 0 durumuna baktı.

Soru (hoca): $N = 0$ verilseydi ne olurdu?

Cevap: $I = 0$ olur, $0 \geq 1$ sağlanmadığı için döngüye girilmez. `sonuc` değişkenine daha önce 1 atandığı için sonuç 1 çıkar.

> **Ek bilgi (derste anlatılmadı):** Tanım gereği $0! = 1$ alınması büyük olasılıkla şu bağıntıdan gelir: $n! = n \times (n-1)!$ eşitliğinin $n = 1$ için de geçerli olması $1! = 1 \times 0!$ yani $0! = 1$ gerektirir. Dolayısıyla "sonuç sıfır olmalı" düşüncesi doğru değildir.

> **Hocanın vurgusu:** Kritik nokta `sonuca` 1 atanan yerdir. Bu atama döngüden önce yapılmasaydı bu hesapların hiçbiri doğru çalışmazdı.

Soru (hoca): $N = -1$ verilseydi?

Cevap: Döngüye yine girilmez, sonuç yine 1 çıkar. Bu istenen bir sonuç değildir.

Gerçekten ihtiyaç duyuluyorsa akışın başına, sayının sıfır ya da sıfırdan büyük olduğunu denetleyen, değilse hatalı giriş olduğunu söyleyen bir kontrol konabilir. Hoca, burada kullanıcının doğru değer girdiğinin kabul edildiğini, bu yüzden bu kontrole gerek olmadığını söyledi. Ama kullanıcının 0 ya da 1 girme olasılığı vardır, algoritma bu değerler için de doğru sonuç üretmelidir.

5.7 Son Değeri 2 Yapmak

Bir öğrenci, 1 ile yeniden çarpmaya gerek olmadığı için döngünün 1 yerine 2'de bitirilmesini önerdi. Bu sırada kısa bir yanlış anlama oldu: başka bir öğrenci 0 verildiğinde sonucun 2 çıkacağını söyledi, ama o `sonucun` ilk değerinin 2 olacağını sanmıştı. Öneri döngünün bitiş değeriyle ilgilidir. Hoca iki uç durumu izledi:

- $N = 0$: $0 \geq 2$ sağlanmaz, sonuç 1 olur.
- $N = 1$: $1 \geq 2$ sağlanmaz, sonuç 1 olur.

Demek ki bir sorun yoktur ve bir işlem azalmış olur.

5.8 Artan For ile Çözüm ve Koşulun Yönü

Soru: `I`'yı `N`'den başlattık. Koşulun büyük eşit mi, küçük eşit mi olduğunu neye göre anlayacağız? -1 adımından dolayı mı büyük eşit?

Cevap: Evet. For döngüsüne burada tersten başladık. Normalde for genelde artımla kullanılır, ama eksiltmenin kullanıldığı yerler de vardır.

Hoca bunu göstermek için döngüyü tersten yazdı: `I` 1'den başlar, birer artar, `N`'e kadar gider, işlem yine `sonuc = I * sonuc` olur, `sonuc` yine 1'den başlar.

```
1. START
2. Girdi: N
3. sonuc = 1
4. for I ← 1, adım 1, son değer N:
     sonuc = I * sonuc
5. Çıktı: sonuc
6. STOP
```

Bir öğrenci sıfır faktöriyelin ayrıca tanımlanması gerekip gerekmediğini sordu, çünkü for içine girilmez. Hoca "girmesin" dedi. `sonuc` zaten 1 olduğu için 0 için de doğru sonuç çıkar. Başlangıç 2 yapılsa da bir sıkıntı olmaz, 1 verilse de algoritma doğru çalışır.

> **Hocanın vurgusu:** For döngüsünü tasarlarken başlangıç ve bitiş noktalarını artım ya da azaltıma göre belirlersiniz. Artım yapıyorsanız başlangıç küçük, bitiş büyük olur. Azaltım yapıyorsanız başlangıç büyük, bitiş küçük olur.

5.9 Atama Okunun Yönü

Bir öğrencinin ok yönüyle ilgili sorusu üzerine hoca bir noktanın çok kritik olduğunu söyledi. Atama her zaman sağdan sola yapılır. `I ← 1` yazılır, çünkü başlangıç değeri `I`'ya atanır.

> **Hocanın vurgusu:** Ok yönü hiçbir zaman tersi olmayacak. `I`'dan 1'e doğru bir ok çizilemez. `I` sabite atanmaz, sabit değişkene atanır.

6. While Döngüsünün Notasyonu

Hoca ardından while döngüsünün gösterimini çizdi. Örnekteki koşul `X > Y` idi. Bu koşul sağlandığı sürece döngü devam eder, sağlanmadığı zaman biter. Yine numaralı noktalar üzerinden gitti:

1. While döngüsüne giriş.
2. While döngüsünden çıkış.
3. Döngü içinde her tura başlangıç.
4. Döngü içinde yapılacak işlerin olduğu kutu. Buraya yalnızca işlem kutusu konmak zorunda değildir. Bir döngünün içine if de, başka bir döngü de konabilir.
5. Turun tamamlanması.
6. Koşul.

Koşul bir tane olabilir. Hoca, ilerleyen zamanda "X büyüktür Y ve aynı zamanda A eşittir B olduğu sürece" gibi birden fazla koşul da yazılacağını belirtti.

Soru (hoca): $X = 5$, $Y = 7$ verilseydi döngü kaç defa dönerdi?

Cevap: Hiç dönmezdi. $5 > 7$ sağlanmadığı için döngüye hiç girilmez.

Soru (hoca): $X = 7$, $Y = 5$ verilseydi?

Cevap: Bir kez. $7 > 5$ sağlanır, içeri girilir. X 6, Y 6 olur. $6 > 6$ sağlanmaz ve döngüden çıkılır.

7. Bir Sayının Basamak Sayısı

Problem: verilen bir sayının kaç basamaklı olduğunu bulan algoritmanın akış diyagramını çiziniz.

7.1 Örnekler ve Fikir

Hoca önce algoritma değil, örnek istedi. 1453 sayısı 4 basamaklıdır, 10374 sayısı 5 basamaklıdır, 1 sayısı 1 basamaklıdır, 0 sayısı da 1 basamaklıdır. Biz buna bakarak cevap verebiliyoruz. Peki bunu bilgisayara nasıl yaptırırız?

Bir öğrenci fikri söyledi: basamak sayısını tutan bir değişken ve kullanıcıdan alınan girdi gerekir. Girdi tekrar tekrar `DIV 10` işlemine sokulur ve her adımda sayaç artırılır. Sayı bitene kadar bu sürer. Hoca bunu "girilen sayıyı ona bölerek bu işi yapacağız" diye özetledi.

7.2 Neden While?

Hoca burada adım sayısının belli olmadığını, bu yüzden while kullanılacağını söyledi. 1453 için 4, 10374 için 5, 1 için 1 bulunur. Her sayıda sonuca farklı bir adımda ulaşılır.

Soru: Sayıya göre belli olmuş olmuyor mu? 1453 verildiyse 4 belli değil mi?

Cevap: Siz bakarak biliyorsunuz, ama bilgisayar 1453'ün 4 basamaklı olduğunu bilmiyor. Faktöriyelde o işi n defa yapacağımızı biliyorduk. Burada bu işi kaç defa yapacağımızı bilmiyoruz. Aradaki fark budur.

7.3 Akış Diyagramı

Dışarıdan sayı alınır, `basamak` adlı bir değişken oluşturulur ve ilk değeri 1 yapılır. Döngü içinde her seferinde `N = N DIV 10` ve `basamak = basamak + 1` denir. Hoca, dar alanda sıkıştığı için kötü yazdığını, öğrencilerin bunu düzgün yazmasını istedi. Tartışmanın vardığı son biçim şöyledir:

```
1. START
2. Mesaj: "N sayısını giriniz"
3. Girdi: N
4. basamak = 1
5. while N >= 10:
     N = N DIV 10
     basamak = basamak + 1
6. Çıktı: basamak
7. STOP
```

7.4 Koşulu Bulmak

Asıl tartışma döngü koşulunda yaşandı.

- **`N < 1`:** Bir öğrencinin ilk önerisiydi. 1453 girilseydi $1453 < 1$ sağlanmaz, döngüye hiç girilmezdi. Koşulun yönü doğru değildir, büyüktür olmalıdır.
- **`N > 10`:** Sonraki öneri buydu ve analiz edildi. Analizde `N` ve `basamak` izlenir.

Burada izlenecek bir indis gerekmiyor, ama bu while döngüsünde hiç indis kullanılmayacağı anlamına gelmez.

$N = 12$ için `N > 10` koşuluyla: $12 > 10$ sağlanır, N 1 olur, `basamak` 2 olur. $1 > 10$ sağlanmaz, çıkılır. Sonuç 2'dir, doğru.

$N = 5$ için: $5 > 10$ sağlanmaz, döngüye girilmez, `basamak` 1 kalır. Bir öğrenci burada sonuç alınamayacağını düşünmüştü, ama sonuç ekrana yazıldığı için 1 çıkar ve bu doğrudur.

$N = 10$ için: Bir öğrenci uç noktayı yakaladı. $10 > 10$ sağlanmaz, sonuç 1 çıkar. Bu yanlıştır, çünkü 10 iki basamaklıdır. Demek ki koşul `N >= 10` olmalıdır. Hoca bunun `N > 9` olarak da yazılabileceğini söyledi.

Son koşulla hocanın örnekleri şöyle izlenir (ara adım eklendi):

N	`N` değerinin sırası	`basamak`
1453	1453, 145, 14, 1	4
10374	10374, 1037, 103, 10, 1	5
12	12, 1	2
10	10, 1	2
5	5	1
1	1	1
0	0	1

> **Hocanın vurgusu:** Bunlar çok önemli. Basit bir büyüktür ya da küçüktür yüzünden bütün program yanlış çalışabilir. Olaya farklı sınır noktaları koyarak çözüm getirebilirsiniz, bunların işlem açısından büyük bir farkı yoktur. Ama sınır değerleri doğru koymazsanız program bir ya da birkaç değer için yanlış çalışır.

7.5 Basamak 0'dan Başlarsa

Soru: `basamak = 0` yapıp koşulu `N >= 1` yazsak olur mu?

Cevap: Hoca önce kontrol etmesi gerektiğini, bunun bir bakış açısı olduğunu söyledi. Bir öğrenci kendisinin bunu denediğini, 0 ya da 1 verildiğinde basamak sayısının 0 çıktığını gördüğünü söyledi. Hoca bunu ekranda `N > 1` ve `basamak = 0` olarak yazdı: 0 girildiğinde $0 > 1$ sağlanmaz, `basamak` 0 çıkar. Oysa 0 sayısı 1 basamaklı kabul edilmişti, yani sonuç yanlıştır. Bu yüzden `basamak = 1` ve `N >= 10` çözümüne geri dönüldü. Hoca bu çözümün 0 ve pozitif sayılar için doğru sonuç ürettiğini söyledi.

> **Ders sonrası not:** Derste iki biçim birbirine karıştı. Büyük olasılıkla öğrencinin sorusu `N >= 1`, ekrandaki biçim ise `N > 1` idi. Bu iki biçim aynı davranmaz. `basamak = 0` ve `N > 1` ile 1 için de 0 çıkar, 12 için de 1 çıkar (12, 1 dizisinde tek tur dönülür), yani bu biçim büyük olasılıkla çok sayıda girdide yanlıştır. `basamak = 0` ve `N >= 1` ile ise 1 için 1, 12 için 2 çıkar; bu biçim büyük olasılıkla negatif olmayan sayılarda yalnız 0 için yanlıştır.

8. Negatif Sayılar ve Alternatif Koşul

Çözüm negatif sayılar için işe yaramıyor gibi görünüyor. Hoca ne yapılması gerektiğini sordu ve iki yaklaşım çıktı.

Birinci yaklaşım: Sayının sıfırdan küçük olup olmadığı başta kontrol edilir. Negatifse -1 ile çarpılıp pozitife çevrilir, sonra while döngüsüne sokulur.

İkinci yaklaşım: Koşul `N ≠ 0` yapılır. Sayı sıfıra eşit olmadığı sürece ona bölünür. Bu koşul hem pozitif hem negatif sayılar için çalışır. Hoca önce fark olmadığını, ikisinde de bir koşul gerektiğini ve birincide belki bir çarpma fazla yapılacağını söyledi. Öğrenci, kendi koşulunun iki yönde de çalıştığını hatırlatınca bunu alternatif çözüm olarak ekrana yazdı.

Önce `basamak = 1` ile denedi. Yazdığı parça şuydu:

```
basamak = 1
while N ≠ 0:
    N = N DIV 10
    basamak = basamak + 1
Çıktı: basamak
```

$N = -12$ için izleme:

- `basamak` 1 ile başlar.
- $-12 \neq 0$ sağlanır. -12 `DIV` $10 = -1$ olur, `basamak` 2 olur.
- $-1 \neq 0$ sağlanır. -1 `DIV` $10 = 0$ olur, `basamak` 3 olur.
- $0 \neq 0$ sağlanmaz, çıkılır. Sonuç 3'tür, bu yanlıştır.

> **Ek bilgi (derste anlatılmadı):** Hocanın burada kullandığı -12 `DIV` $10 = -1$ ve -1 `DIV` $10 = 0$ değerleri sıfıra doğru kesmedir. Sonraki derslerde görülecek C dilinde tam sayı bölmesi de büyük olasılıkla aynı sonucu verir (`gcc` ile denendi: `-12 / 10` için -1, `-1 / 10` için 0). Bazı dillerde ise bölme aşağıya yuvarlar. Örneğin Python'da `-12 // 10` sonucu -2'dir. Bu yüzden negatif sayılarda sonuç büyük olasılıkla kullanılan dile bağlıdır.

Bir öğrenci çözümü gösterdi: `basamak` ilk değeri 0 yapılmalı, ama bu durumda 0 sayısı için ayrı bir koşul yazılmalı. Hoca da görülmesini istediği noktanın bu olduğunu söyledi. `basamak` 0'dan başlatıldığında bütün sayılar için doğru çalışır, ama 0 için çalışmaz, çünkü 0 girildiğinde döngüye girilmez ve 0 çıkar. Yine ekstra bir kontrole ihtiyaç vardır.

Hoca bu parçaların tamamlanmış akışlar olmadığını, yalnızca alternatifleri görmek için yazıldığını özellikle belirtti.

> **Hocanın vurgusu:** Bu tür eksik akışları çizmeyin. Akış diyagramında START, STOP ve geri kalan bütün kutular bulunmalıdır. Bunlar yalnızca alternatif çözümleri göstermek içindir.

Dersin Toparlaması

Bu derste önce üçgen türü problemi üzerinden karşılaştırmaların ikili olduğu, iç içe if kollarının nasıl birleştirildiği ve gereksiz kontrollerden nasıl kaçınıldığı görüldü. Dik üçgen eklenince kod uzunluğu ile işlem sayısının farklı şeyler olduğu ortaya çıktı: önce doğru algoritma tasarlanır, sonra kontrol sayısı azaltılır. Ardından döngülere geçildi. Adım sayısı belli olan işlerde for, belli olmayan işlerde while kullanılır ve adım sayısının girdiye bağlı olması onu belirsiz yapmaz. Faktöriyel bu yüzden for ile, basamak sayısı ise while ile çözüldü. İki örnekte de ortak ders şudur: birikim değişkenine döngüden önce doğru ilk değer verilmeli ve algoritma 0, 1, 10, negatif sayı gibi uç değerlerle analiz edilmelidir.

Hoca sonraki derste for ve while ile örnek problemlere devam edileceğini söyledi. Ardından karakterlerle ilgili işler yapılacak ve string konusu konuşulacak, sonra dizilere geçilecek. Dizilerden sonra C programlama diliyle ilgili temel bilgiler verilmeye başlanacak.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.